REVERSE FALLS — DECISION MAP / CANON STORY SYSTEM

Нужно расширить существующую «Карту решений» и одновременно привести систему сохранений к архитектуре, которая будет безопасно работать в долгой игре примерно на 60 глав.

Главная цель:

У игрока всегда должна существовать ОДНА официальная текущая история.

Все альтернативные прохождения, просмотренные ветки и старые решения должны храниться отдельно и НИКОГДА не путаться с текущим каноном.

Это критически важно для будущих глав.

Не делай временное решение только для Chapter 0 и Chapter 1.

Система должна быть рассчитана на десятки глав, сотни выборов и последствия, возвращающиеся намного позже.

---

1. ГЛАВНЫЙ ПРИНЦИП

В проекте должен существовать один authoritative / canonical story state.

Он является:

SINGLE SOURCE OF TRUTH

для текущей истории игрока.

Никогда не определяй текущий канон на основе:

- просмотренных сцен;
- discovered nodes;
- ранее сыгранных альтернатив;
- истории посещённых веток;
- UI карты;
- старых сохранений;
- cached state.

Только canonical state определяет:

что реально произошло в текущей истории.

---

2. ОБЯЗАТЕЛЬНО РАЗДЕЛИТЬ 4 ТИПА ДАННЫХ

Не смешивать их.

A. CANON STORY STATE

Официальная текущая история.

Например:

```
{
  "schemaVersion": 1,
  "currentChapter": 2,
  "mainPath": {
    "chapter0": {
      "creature": "vampire"
    },
    "chapter1": {
      "gideonReason": "openness",
      "tracksChoice": "inspect_area",
      "slopeChoice": "pacifica_first",
      "endingChoice": "observe"
    }
  },
  "relationships": {
    "pacifica_gideon": 2
  },
  "storyFlags": {
    "extra_clue_01": true,
    "journal_doubt": false
  }
}
```

Это единственный источник правды для будущего сюжета.

B. DISCOVERY STATE

Отдельно хранится всё, что игрок когда-либо открыл.

Например:

```
{
  "discoveredNodes": [],
  "discoveredChoices": [],
  "seenScenes": [],
  "seenEndings": []
}
```

Discovery state НЕ определяет текущую историю.

Пример:

Игрок раньше прошёл:

Werewolf

потом изменил канон на:

Vampire.

Тогда:

CANON:

`vampire`

DISCOVERY:

`werewolf` `vampire`

Werewolf остаётся открытым на карте, но больше не считается произошедшим событием.

C. ALTERNATIVE SESSION STATE

Временное состояние режима:

«Посмотреть альтернативу»

Оно должно быть полностью изолировано от основного сохранения.

Например:

```
{
  "active": true,
  "originSaveId": "...",
  "startNode": "...",
  "temporaryChoices": {},
  "temporaryFlags": {}
}
```

Когда игрок выходит из альтернативного режима:

временный state можно уничтожить.

Основной канон НЕ должен меняться.

D. SAVE METADATA

Техническая информация сохранения:

- save ID;
- timestamp;
- current chapter;
- scene ID;
- schema version;
- game version;
- checkpoint;
- save format version.

Не смешивать её со story flags.

---

3. НОВЫЙ РЕЖИМ КАРТЫ

Сейчас есть:

Мой путь

Вся схема

Добавить:

ПОЛНЫЙ РЕЖИМ

Получится:

Мой путь | Полный режим | Вся схема

---

4. МОЙ ПУТЬ

Показывает только текущий канонический путь.

Например:

Chapter 0

→ Vampire

Chapter 1

→ Gideon openness

→ Inspect area

→ Pacifica first

→ Observe

Chapter 2

→ ...

Это реальная история текущего сохранения.

В этом режиме нельзя случайно изменить прошлое обычным кликом.

---

5. ВСЯ СХЕМА

Показывает структуру уже доступной части игры:

- развилки;
- объединения;
- открытые альтернативы;

- найденные сцены;
- discovered nodes.

Это обзор структуры.

Простой просмотр здесь никогда не должен изменять save state.

---

6. ПОЛНЫЙ РЕЖИМ

Позволяет взаимодействовать с уже открытыми решениями.

При выборе альтернативного решения игрок получает:

Посмотреть альтернативу

или

Изменить основной путь

Никогда не менять канон сразу после клика по узлу.

---

7. ПОСМОТРЕТЬ АЛЬТЕРНАТИВУ

Это безопасный режим «Что если».

Например:

Current canon:

`vampire`

Игрок выбирает:

`werewolf`

и нажимает:

Посмотреть альтернативу

Создаётся isolated temporary story state.

Все новые решения применяются только к нему.

Нельзя изменять:

- canonical mainPath;
- canonical relationships;
- canonical storyFlags;
- canonical inventory;
- canonical quest state;
- canonical chapter progress.

Допускается обновлять только discovery state:

например, если игрок впервые увидел новую сцену.

---

8. В ИГРЕ ДОЛЖНО БЫТЬ ВИДНО, ЧТО ЭТО АЛЬТЕРНАТИВА

Во время альтернативного прохождения показывать небольшой индикатор:

Альтернативный путь

Не большой баннер.

Но игрок должен всегда понимать:

это НЕ его текущий канон.

---

9. ВОЗВРАТ ИЗ АЛЬТЕРНАТИВЫ

После выхода:

- восстановить основной save;
- вернуть canonical state;
- удалить temporary alternative state;
- вернуть игрока туда, где он находился до входа в альтернативу, либо в карту решений.

Никакие временные флаги не должны протекать в основной save.

---

10. ИЗМЕНИТЬ ОСНОВНОЙ ПУТЬ

Это уже реальное изменение истории.

Игрок выбирает старое решение и нажимает:

Изменить основной путь

Показать подтверждение:

Изменить историю?

Вы собираетесь изменить решение в предыдущей главе.

Решения и состояния после этой точки, зависящие от старого пути, будут пересчитаны или сброшены.

Новое решение станет частью вашей основной истории.

Отмена

Изменить путь

---

11. ПЕРЕД ИЗМЕНЕНИЕМ ОБЯЗАТЕЛЬНО СОЗДАТЬ CHECKPOINT

Перед mutation canonical state:

создать recovery checkpoint.

Например:

```
pre_rewrite_checkpoint
```

Если что-то сломалось во время пересчёта:

полностью восстановить старый canonical state.

Никаких частично изменённых сейвов.

---

12. ИЗМЕНЕНИЕ ДОЛЖНО БЫТЬ АТОМАРНЫМ

Нельзя делать:

1. изменить creature;
2. сохранить;
3. удалить старые flags;
4. сохранить;
5. пересчитать chapter;
6. ошибка.

Это может оставить повреждённую историю.

Нужно:

1. загрузить старый state;
2. клонировать его;
3. применить изменение к копии;
4. пересчитать зависимости;
5. валидировать;
6. только если всё валидно — заменить canonical state;
7. сохранить один согласованный результат.

Если validation не прошёл:

ничего не менять.

---

13. ЗАВИСИМОСТИ ВЫБОРОВ

Каждое важное решение должно иметь стабильный ID.

Например:

```
ch0.creature
ch1.gideon_reason
ch1.tracks
ch1.slope
ch1.ending
```

Не использовать текст реплики как ID.

Потому что текст позже может измениться или переводиться.

---

14. У КАЖДОГО CHOICE НУЖНЫ STABLE OPTION IDS

Например:

```
choiceId: ch0.creature

options:
werewolf
wendigo
vampire
```

А не:

```
"Выбрать вампира"
```

как системный ключ.

UI-текст должен быть отдельно от логики.

---

15. НЕ ХАРДКОДИТЬ ЛОГИКУ В UI

Карта решений должна только визуализировать story graph.

Она НЕ должна сама решать:

какие флаги удалить; какие сцены каноничны; как пересчитать Chapter 20.

Это должна делать отдельная story-state / dependency система.

UI вызывает:

например:

```
rewriteCanonFrom(choiceId, newOptionId)
```

а не самостоятельно меняет JSON.

---

16. STORY GRAPH

Использовать data-driven структуру.

Каждый узел должен иметь примерно:

```
{
  "nodeId": "ch1.tracks.inspect_area",
  "chapterId": "chapter1",
  "choiceId": "ch1.tracks",
  "optionId": "inspect_area",
  "parents": [],
  "children": [],
  "requirements": [],
  "effects": [],
  "rejoinNode": null
}
```

Не обязательно именно такой формат.

Но архитектура должна поддерживать:

- branching;
- merging;
- requirements;
- effects;
- dependencies;
- future callbacks.

---

17. ВЕТКИ ДОЛЖНЫ УМЕТЬ СНОВА СОЕДИНЯТЬСЯ

Наша игра НЕ состоит из сотен полностью отдельных сюжетов.

Например:

```
        B1
A ---- B2 ---- C
        B3
```

После B1/B2/B3 сюжет снова приходит к C.

Но разные flags могут сохраниться.

Это базовая архитектура всей игры.

---

18. ПРИ ИЗМЕНЕНИИ СТАРОГО ВЫБОРА НЕЛЬЗЯ БЕЗДУМНО УДАЛЯТЬ ВСЁ ПОСЛЕ

Пример:

Chapter 0:

Creature = Vampire

Chapter 1:

Gideon openness

Эти два решения могут быть независимыми.

Если игрок меняет:

Vampire → Wendigo

не обязательно удалять Gideon openness.

Нужно использовать dependency information.

Удалять/пересчитывать только состояние, которое действительно зависит от изменённого решения.

---

19. НО БЕЗОПАСНОСТЬ ВАЖНЕЕ

Если dependency невозможно надёжно определить:

лучше сбросить историю до ближайшего безопасного checkpoint / начала главы.

Не оставлять потенциально противоречивое состояние.

---

20. STORY FLAGS

У каждого story flag желательно знать происхождение.

Например:

```
{
  "extra_clue_01": {
    "value": true,
    "source": "ch1.tracks.inspect_area"
  }
}
```

Тогда при изменении решения система знает:

почему появился флаг.

Это намного безопаснее, чем огромный плоский объект boolean-значений без происхождения.

---

21. ОТНОШЕНИЯ ПЕРСОНАЖЕЙ

Relationship changes тоже должны быть traceable.

Например:

```
{
  "pacifica_gideon": {
    "base": 0,
    "modifiers": [
      {
        "source": "ch1.gideon_reason.openness",
        "value": 1
      },
      {
        "source": "ch1.slope.pacifica_first",
        "value": 1
      }
    ]
  }
}
```

Final value вычисляется из modifiers.

Тогда изменение старого решения может удалить только соответствующий modifier.

Не делать только:

```
pacifica_gideon = 14
```

без понимания, откуда взялись эти 14.

Это критически важно для долгой игры.

---

22. INVENTORY / CLUES

То же касается предметов и улик.

Например:

```
dark_cloth
```

должен знать:

```
source = ch1.tracks.inspect_area
```

Если canonical choice изменился на другую ветку:

предмет можно корректно удалить.

---

23. QUEST / KNOWLEDGE STATE

В будущем персонажи могут знать разную информацию.

Поэтому отдельно поддержать knowledge flags.

Например:

```
pacifica_knows_x
gideon_knows_y
wendy_knows_journal
```

Не предполагать автоматически, что если игрок видел сцену, то персонаж её тоже видел.

Discovery state игрока ≠ knowledge state персонажей.

---

24. НЕ ПУТАТЬ PLAYER KNOWLEDGE И CHARACTER KNOWLEDGE

Это очень важно.

Игрок в альтернативной ветке может увидеть секрет.

Но в основном каноне Пасифика этого секрета не знает.

Следовательно:

просмотр alternative session:

может добавить:

`playerDiscoveredSecret = true`

но НЕ:

`pacificaKnowsSecret = true`

в canonical state.

---

25. СХЕМА ВЕРСИЙ

Добавить:

```
schemaVersion
```

в save/story state.

В будущем структура сохранений будет меняться.

Нужна migration system.

Например:

```
v1 -> v2
v2 -> v3
```

Не ломать старые сохранения после будущих обновлений.

---

26. НИКОГДА НЕ ИЗМЕНЯТЬ ID ПОСЛЕ РЕЛИЗА

После появления choice/node/flag ID в публичной версии:

не переименовывать его без migration.

Например:

если был:

`ch1.tracks.inspect_area`

нельзя просто изменить на:

`chapter1.search_area`

иначе старые сохранения сломаются.

Если rename нужен:

делать migration.

---

27. СОХРАНЕНИЯ ДОЛЖНЫ ВАЛИДИРОВАТЬСЯ

После загрузки проверить:

- schema version;
- required fields;
- known chapter IDs;
- known choice IDs;
- valid option IDs;
- conflicting mutually-exclusive flags;
- invalid current scene;
- orphaned relationship modifiers;

- orphaned inventory sources.

Если найдено повреждение:

не молча продолжать игру.

Попробовать repair / rollback.

---

28. MUTUALLY EXCLUSIVE CHOICES

Система должна понимать, что:

```
werewolf
wendigo
vampire
```

в одном canonical path не могут одновременно быть true.

Нельзя хранить:

```
werewolf=true
vampire=true
```

как canonical state.

Хранить:

```
ch0.creature = vampire
```

---

29. DISCOVERY ПРИ ЭТОМ МОЖЕТ СОДЕРЖАТЬ ВСЕ ТРИ

Например:

```
discovered:
werewolf
```

```
wendigo
vampire
```

это нормально.

Но canonical choice всё равно один.

---

30. КАРТА РЕШЕНИЙ — ЦВЕТА

Сохранить текущий дизайн.

Добавить визуальные состояния:

Основной путь

существующий голубой / cyan.

Просматриваемая альтернативная ветка

золотой.

Открытая, но не текущая ветка

серый / приглушённый.

Не открыто

существующий стиль locked/unopened.

Сейчас

отдельный highlight.

Добавить понятную легенду.

---

31. ИНФОРМАЦИЯ О ВЫБОРЕ

При выборе узла в нижней панели показывать:

Название сцены.

Краткое описание.

Глава.

Если это choice:

Ваш основной выбор: X

Открытые варианты:

A
B
C

Можно изменить основной путь: Да

Не раскрывать неизвестные последствия.

---

32. НИКАКИХ БУДУЩИХ СПОЙЛЕРОВ

Если решение Chapter 2 влияет на Chapter 28:

не писать это заранее.

Показывать только последствия, которые игрок уже увидел.

---

33. ФУНКЦИЯ «ПОКАЗАТЬ СПОЙЛЕРЫ»

Существующая кнопка может показывать скрытую структуру только в рамках текущих правил игры.

Но «Полный режим» сам по себе НЕ должен автоматически раскрывать весь будущий контент.

---

34. ИЗМЕНЕНИЕ РЕШЕНИЯ ИЗ CHAPTER 0

Обязательно протестировать:

```
Werewolf -> Wendigo
Werewolf -> Vampire
Wendigo -> Werewolf
Wendigo -> Vampire
```

```
Vampire -> Werewolf
Vampire -> Wendigo
```

После этого новый Chapter 1 должен читать правильный canonical creature.

---

35. CHAPTER 1

Все новые выборы Chapter 1 должны быть зарегистрированы в общей системе.

Не делать отдельную bespoke систему только для этой главы.

Например:

```
ch1.gideon_reason
ch1.tracks
ch1.slope
ch1.ending
```

---

36. CURRENT CANON SUMMARY

Очень желательно создать функцию:

```
getCanonSummary()
```

которая формирует структурированный summary текущего прохождения.

Это пригодится будущим главам и разработке.

Например:

```
{
  "currentChapter": 12,
  "importantChoices": {},
  "relationships": {},
  "activeStoryFlags": {},
  "inventory": {},
  "characterKnowledge": {}
}
```

---

37. ОТДЕЛЬНЫЙ CANON STORY STATE

Если архитектура проекта позволяет, создать отдельный файл / storage object:

```
canon_story_state
```

Название может быть другим.

Но логически должен существовать отдельный объект.

Главное правило:

Treat canon story state as the single source of truth for the player's current story.

Never infer current canon from discovered nodes, previously played branches, alternative sessions or decision-map history.

---

38. FUTURE CODEX WORK

Когда в будущем Codex реализует Chapter 20, он должен иметь возможность спросить story state:

например:

```
getChoice("ch0.creature")
hasFlag("monster_spared")
getRelationship("pacifica_gideon")
characterKnows("wendy", "journal3")
hasItem("dark_cloth")
```

А не вручную разбирать 20 старых глав.

---

39. API / STORY STATE SERVICE

Если проект позволяет, централизовать работу с сюжетом.

Например:

```
StoryState.getChoice()
StoryState.setChoice()
StoryState.hasFlag()
StoryState.getRelationship()
StoryState.hasItem()
StoryState.characterKnows()
StoryState.createAlternativeSession()
StoryState.commitCanonRewrite()
StoryState.rollback()
```

Не разбрасывать прямую запись save JSON по десяткам файлов.

---

40. НИКАКИХ ПРЯМЫХ MUTATIONS ИЗ СЦЕН

Предпочтительно:

```
StoryState.applyEffect(...)
```

а не:

```
save.relationships.pacifica_gideon += 1
```

из каждого отдельного сценарного файла.

Это позволит потом безопасно пересчитывать историю.

---

41. TRANSACTION LOG

Если возможно, сохранять небольшой internal log story mutations.

Например:

```
choice ch1.tracks = inspect_area
added flag extra_clue_01
added item dark_cloth
```

Это поможет:

- debugging;
- rollback;
- migrations;
- поиск багов.

Не обязательно показывать игроку.

---

42. BACKUPS

Перед:

- canon rewrite;
- migration;
- крупным save conversion;

создавать recovery backup.

Ограничить количество старых backup-файлов, чтобы не засорять storage.

---

43. AUTO-REPAIR

Если при загрузке обнаружена мелкая проблема:

например orphaned derived flag,

можно пересчитать derived state из canonical choices.

Лучше recompute derived state, чем доверять старому кэшу.

---

44. DERIVED STATE

Если значение можно безопасно вычислить из истории:

не обязательно считать его independent truth.

Например relationship total лучше получать из modifiers.

Canonical choices являются первичными.

Derived values могут пересчитываться.

---

45. ТЕСТ НА УТЕЧКУ ALTERNATIVE STATE

Обязательно:

1. Основной путь = Vampire.
2. Запустить alternative Werewolf.
3. Получить альтернативную улику.
4. Изменить альтернативное relationship.
5. Выйти.
6. Перезагрузить игру.

Проверить:

- canonical creature всё ещё Vampire;
- улика из alternative отсутствует;
- relationship не изменился;
- альтернативная сцена может остаться discovered.

---

46. ТЕСТ НА REWRITE

1. Canon = Vampire.
2. Пройти Chapter 1.
3. Получить flags/items/relationships.
4. Из карты изменить Chapter 0 на Wendigo.
5. Выполнить rewrite.
6. Проверить dependency graph.
7. Проверить отсутствие противоречивых флагов.
8. Сохранить.
9. Закрыть игру.
10. Запустить снова.
11. Проверить canonical story.

---

47. ТЕСТ НА НЕЗАВИСИМЫЙ ВЫБОР

Если поздний выбор не зависит от изменённого старого:

он должен сохраниться.

Не сбрасывать всё автоматически, если это не требуется.

---

48. ТЕСТ НА ЗАВИСИМЫЙ ВЫБОР

Если выбор больше невозможен из-за нового прошлого:

его обязательно удалить или переиграть.

Никаких impossible states.

---

49. ТЕСТ НА СТАРЫЕ SAVE

Проверить существующие сохранения Chapter 0 / Chapter 1.

Новая архитектура не должна уничтожать старый прогресс.

Если формат изменился:

создать migration.

---

50. ТЕСТ НА NEW GAME

Полностью новое прохождение тоже должно работать без legacy data.

---

51. ТЕСТ НА MULTIPLE SAVE SLOTS

Если игра поддерживает несколько saves:

каждый slot должен иметь собственный:

- canonical state;
- discovery state;
- chapter progress.

Не смешивать данные разных прохождений.

Если discovery задуман глобальным для профиля, это должно быть явное архитектурное решение, а не случайность.

---

52. НЕ ПЕРЕПИСЫВАТЬ РАБОЧИЙ UI БЕЗ НУЖДЫ

Существующая карта визуально хорошая.

Сохрани:

- стиль;
- масштабирование;
- перемещение;
- вкладки глав;
- panel description;
- current-path rendering;
- spoilers option.

Расширить аккуратно.

---

53. НЕ ДЕЛАТЬ БЫСТРЫЙ ХАК

Не делай систему вида:

```
if chapter == 0:
  ...
if chapter == 1:
  ...
```

Мы будем добавлять много глав.

Используй generic data-driven system.

---

54. ДОКУМЕНТАЦИЯ

После реализации создай короткую техническую документацию внутри проекта:

например:

```
STORY_STATE_ARCHITECTURE.md
```

В ней описать:

- canonical state;

- discovery state;
- alternative state;
- choice IDs;
- node IDs;
- effects;
- dependencies;
- migrations;
- canon rewrite;
- как будущая глава должна читать story state;
- как добавлять новый выбор безопасно.

Это обязательно.

Потому что будущие Codex-сессии должны понимать архитектуру и не ломать её.

---

55. ДОБАВИТЬ RULES ДЛЯ БУДУЩИХ ГЛАВ

В документации явно записать:

1. Never infer canon from discovery.
2. Never directly edit canonical JSON from UI.
3. Every choice needs a stable ID.
4. Every option needs a stable ID.
5. Every persistent effect should have a source.
6. Relationship changes should be traceable.
7. Items/clues should have sources.
8. Alternative sessions must never write to canonical state.
9. Canon rewrites must be transactional.
10. Save schema changes require migrations.

---

56. ВАЖНЫЙ ПРИНЦИП ДЛЯ 60-ГЛАВНОЙ ИГРЫ

Не нужно хранить 60 глав как одну гигантскую ветку с миллионами комбинаций.

Хранить:

- основной набор canonical choices;
- persistent flags;
- relationship modifiers;
- inventory;
- character knowledge;
- ключевые consequences.

Сюжетные ветви могут снова сходиться.

Следующие главы читают только необходимые им прошлые состояния.

---

57. PERFORMANCE

Карта решений может стать большой.

Не рендерить без необходимости тысячи невидимых элементов.

Подумать о:

- chapter-based loading;
- lazy graph rendering;
- caching layout;
- сохранении текущего zoom/pan.

Но не усложнять преждевременно.

Сейчас главное — хорошая архитектура данных.

---

58. ERROR HANDLING

Если rewrite невозможен:

не молча ломать save.

Показать понятное сообщение и восстановить checkpoint.

Например:

Не удалось изменить историю. Ваше сохранение не было изменено.

---

59. FINAL INVARIANTS

После любой операции всегда должно быть true:

Invariant 1

У каждого canonical choice ровно один selected option.

Invariant 2

Canonical state не содержит mutually-exclusive contradictory values.

Invariant 3

Alternative state никогда не изменяет canonical state.

Invariant 4

Discovery не определяет canon.

Invariant 5

Derived state соответствует canonical choices.

Invariant 6

Все persistent effects имеют понятное происхождение.

Invariant 7

Loaded save проходит validation.

Invariant 8

Изменение старого выбора либо полностью успешно, либо вообще не применяется.

---

60. ПОСЛЕ РЕАЛИЗАЦИИ

Не просто скажи «готово».

Дай отчёт:

Architecture

- где хранится canonical state;
- где discovery;
- где alternative state;
- как устроен dependency graph;
- как хранятся IDs;
- как работает relationship calculation.

Files changed

Перечисли изменённые и новые файлы.

Migrations

Опиши migration существующих saves.

Rewrite

Опиши алгоритм изменения старого решения.

Rollback

Как защищается save.

Tests

Перечисли реально выполненные тесты.

Known limitations

Честно перечисли оставшиеся ограничения.

Future chapter guide

Покажи короткий пример того, как Chapter 2/3 должен зарегистрировать новый choice и его effects.

САМОЕ ГЛАВНОЕ

Эту систему мы строим не ради Chapter 0 и Chapter 1.

Мы строим фундамент для игры примерно на 60 глав.

Лучше сейчас потратить больше работы на чистую архитектуру, чем потом исправлять сотни несовместимых флагов.

У каждого save должен всегда существовать один однозначный официальный timeline.

CANON STORY STATE = SINGLE SOURCE OF TRUTH.

Все альтернативы, discovery, просмотренные ветки и эксперименты существуют отдельно.

Не начинай с переписывания UI.

Сначала изучи текущие:

- decision map;
- save/load;
- chapter state;
- flags;
- choice storage;
- Chapter 0;
- Chapter 1.

После этого спроектируй минимально-инвазивную архитектуру поверх существующего проекта, реализуй её, мигрируй существующие saves и только затем подключи новый «Полный режим» к интерфейсу.

Обязательно запусти тесты и саму игру после изменений.